PINKYCORE · DOCUMENTACIÓN TÉCNICA ACCESIBLE

GLITCH

Guía para entender, estudiar y explicar el funcionamiento del código

- ARQUITECTURA
- GAMEPLAY
- IA Y EVENTOS
- PERSISTENCIA

GUÍA DE ESTUDIO

JULIO 2026

Contenido

Una lectura de arriba hacia abajo explica la partida completa. Las últimas secciones funcionan como material de repaso para la exposición.

01 El juego explicado en un minuto

02 La forma más fácil de imaginar la arquitectura

03 Conceptos de Unity que aparecen en el proyecto

04 Recorrido completo de una partida

05 `GameManager`: el director general

06 `PlayerController`: controles, movimiento y defensa

07 `EnemyController`: la anomalía

08 State Hijack: convertir defensa en contraataque

09 Generación procedural y sectores

10 Eventos globales y eventos por mapa

11 Contratos y mejoras durante la run

12 Clases o estilos de juego

13 Tutoriales

14 Interfaz, jerarquía visual y colores

15 Audio

16 Guardado local y metaprogresión

17 Ranking global con Firebase

18 Telemetría local

19 Duelo local

20 Menús y cambios de escena

21 Inventario de scripts por grupo

22 Ejemplo de interacción completa

23 Qué conviene decir si preguntan por decisiones de diseño del código

24 Guion oral de cinco minutos

25 Plan breve para aprenderlo

Esta guía está pensada para estudiar el proyecto y poder explicarlo oralmente. No intenta enseñar cada línea ni reemplazar el código. La idea es entender qué responsabilidad tiene cada sistema, cómo se comunica con los demás y por qué la organización actual permite que una partida funcione de principio a fin.

Estado del proyecto revisado: julio de 2026.

1. El juego explicado en un minuto

GLITCH es un juego de supervivencia y persecución en vista superior. El jugador controla a un corredor atrapado en una instalación de contención. Una anomalía lo persigue de manera constante, cambia de estado y obtiene ataques más complejos a medida que avanza la partida.

El jugador no tiene una vida tradicional. En condiciones normales, un impacto de la anomalía, un proyectil o ciertos peligros termina la run. Para evitarlo dispone de movimiento, Ghost Dash, Firewall Parry, Firewall Burst, escudos, powerups y, más adelante, State Hijack. Mientras sobrevive obtiene puntos, recoge datos, completa objetivos y contratos, elige mejoras temporales y atraviesa Breaches que regeneran la arena como un sector nuevo.

El código está organizado alrededor de una idea sencilla: cada componente se ocupa de una responsabilidad. `PlayerController` controla al jugador; `EnemyController`, a la anomalía; `ProceduralArenaGenerator`, al mapa; `ArenaChaosDirector`, al contenido global de la arena; y `GameManager` coordina la partida completa.

2. La forma más fácil de imaginar la arquitectura

Conviene pensar el juego como un equipo de trabajo:

Sistema	Forma simple de entenderlo
`GameManager`	Es el director de la partida. Decide cuándo empieza, qué está habilitado y cuándo termina.
`PlayerController`	Es el cuerpo y los controles del corredor.
`EnemyController`	Es el cerebro y el cuerpo de la anomalía.
`ProceduralArenaGenerator`	Es el constructor del escenario.
`ArenaChaosDirector`	Es quien llena el escenario con pickups, objetivos y eventos globales.
Controladores de Lab, Storage, Rupture, Core y Archive	Son las reglas particulares de cada tipo de mapa.
`GameMenuController`	Maneja pausa, derrota y resultado.
`MainMenuController`	Maneja el menú principal, las clases, el archivo y las opciones.
Clases terminadas en `Storage`	Guardan información entre una partida y otra.
`GlitchAudioManager`	Recibe pedidos como “sonido de Parry” y resuelve cómo reproducirlos.

La relación principal se puede resumir así:

```
Menú principal
      ↓
Elección de modo y clase
      ↓
Generación de arena ──→ eventos propios del mapa
      ↓
GameManager inicia y coordina la run
      ├── PlayerController lee al jugador
      ├── EnemyController persigue y cambia de estado
      └── ArenaChaosDirector crea recompensas y eventos
      ↓
Derrota o Breach
      ├── Breach: genera un sector nuevo y la run continúa
      └── Derrota: guarda resultado, progresión y telemetría
```

3. Conceptos de Unity que aparecen en el proyecto

No hace falta explicar Unity de manera demasiado técnica. Con estas ideas alcanza:

- Un `GameObject` es un objeto de la escena.
- Un componente es una función agregada a ese objeto. Por ejemplo, el jugador tiene componentes para dibujarse, colisionar, moverse y ejecutar `PlayerController`.
- `MonoBehaviour` significa que el script puede vivir como componente dentro de Unity.
- `Awake` prepara referencias y valores internos antes de empezar.
- `Start` inicia lo que depende de que la escena ya esté armada.
- `Update` se ejecuta una vez por cuadro y se usa para leer controles, avanzar relojes y cambiar estados.
- Un `Rigidbody2D` mueve un objeto mediante la física 2D.
- Un `Collider2D` define el área que puede chocar o detectar contacto.
- Una coroutine permite desarrollar una secuencia a lo largo del tiempo, por ejemplo: avisar, esperar, activar un peligro y luego retirarlo.
- `SerializeField` permite ajustar un valor desde el Inspector sin volver pública toda la lógica.
- `PlayerPrefs` es el guardado local simple que se usa para preferencias y progresión.
- `OnGUI` dibuja gran parte de la interfaz directamente desde código. En este proyecto los menús, el HUD y varias ventanas calculan su tamaño y posición en cada resolución.

Una respuesta oral posible sería: “Unity llama automáticamente a métodos como `Awake`, `Start` y `Update`. Yo reparto el comportamiento en componentes y cada componente usa esos momentos para prepararse, iniciarse o actualizarse”.

4. Recorrido completo de una partida

4.1. Entrada al menú

`MainMenuController` dibuja el menú principal y consulta los sistemas de guardado para mostrar datos reales. Si no existe un nombre válido registrado, abre una ventana obligatoria y bloquea el menú de atrás hasta que el jugador escriba uno. Esto evita usuarios genéricos como “PlayerOne” en el ranking.

Las opciones principales son:

- Jugar, que contiene Contención y Duelo Local.
- Archivo, que contiene desbloqueos, logros y estadísticas.
- Opciones, con audio, interfaz, sistema y tutoriales.
- Créditos como vínculo secundario.

En Contención, el jugador elige una clase. En Duelo Local, el juego marca el modo versus y carga directamente la escena de gameplay. El cambio de escena pasa por `SceneTransitionController`, que cubre la pantalla con un fundido para evitar un corte visual brusco.

4.2. Construcción de la arena

Al cargar gameplay, `ProceduralArenaGenerator` elige un tema permitido y construye:

1. límites exteriores;
2. obstáculos estáticos;
3. obstáculos dinámicos;
4. detalles visuales y fondo;
5. controladores de eventos correspondientes al tema;
6. el director global de pickups y eventos.

El generador conserva carriles transitables y zonas reservadas. No coloca objetos totalmente a ciegas: registra áreas ocupadas para reducir superposiciones y rutas imposibles.

4.3. Preparación e inicio

`GameManager` reinicia los datos de la run, lee la clase seleccionada, comienza la telemetría y prepara tutoriales, contratos, mejoras y dificultad. Durante esta preparación congela el tiempo. Después ejecuta una cuenta regresiva corta de 3, 2, 1 y GO.

Si corresponde mostrar el tutorial inicial, la partida permanece detenida mientras se practican o explican movimiento, Parry, Firewall, recursos y eventos. Las pausas contextuales también congelan la acción, pero solo aparecen la primera vez que se encuentra cada mecánica o evento.

4.4. Run activa

Cuando la fase pasa a `Active` suceden varias cosas al mismo tiempo:

- el jugador puede moverse y usar habilidades;
- la anomalía persigue, navega y cambia de estado;
- el puntaje aumenta por supervivencia;
- aparecen pickups y datos;
- se habilitan contratos y mejoras temporales;
- la dificultad desbloquea estados y eventos por etapas;
- el director controla que no se superpongan demasiadas amenazas grandes;
- el HUD consulta el estado de todos esos sistemas y lo representa.

4.5. Breach o derrota

Una Breach no es simplemente un portal. Primero aparece una advertencia, la anomalía es guiada hacia la brecha, comienza un barrido que consume la arena y el jugador debe llegar al portal. Si entra a tiempo, el generador destruye la geometría anterior, aumenta el número de sector, selecciona otro tema permitido y construye una arena nueva. La persecución continúa con más presión.

Si el jugador recibe un impacto sin defensa, `GameManager` inicia la secuencia de derrota. Se reproduce la explosión visual, se detiene la partida y se entregan los datos correspondientes. `GameMenuController` muestra el puntaje, tiempo, datos y rango final, y registra automáticamente el resultado en el ranking si supera el mejor puntaje personal.

5. `GameManager`: el director general

`GameManager` es el mejor archivo para empezar a explicar el proyecto porque conecta casi todo. Su trabajo no es mover al jugador ni decidir la trayectoria de la anomalía. Su trabajo es coordinar.

Sus responsabilidades principales son:

- controlar las fases de inicio y la cuenta regresiva;
- saber si la run está activa o terminó;
- contar tiempo y puntaje;
- habilitar mecánicas según el tiempo sobrevivido;
- aplicar los efectos de la clase elegida;
- abrir tutoriales;
- crear y seguir contratos;
- ofrecer mejoras temporales;
- regular la dificultad y la presión de eventos;
- recibir avisos de otros sistemas;
- dibujar el HUD;
- cerrar la run y entregar recompensas.

Cómo recibe avisos

Los demás componentes llaman métodos con nombres claros. Por ejemplo:

- al recoger un dato se llama a `NotifyScorePickupCollected`;
- al hacer Parry se llama a `NotifyParrySuccess`;
- al usar Firewall se llama a `NotifyFirewallBurstActivated`;
- al escapar por una Breach se llama a `NotifyBreachEscaped`;
- al comenzar un evento se llama a `NotifyMapEventStarted`.

Con esos avisos el manager actualiza contratos, logros, desafíos diarios, tutoriales y telemetría sin obligar al pickup o al enemigo a conocer todos esos sistemas.

Puntaje

El puntaje base combina:

- 4 puntos por cada segundo sobrevivido;
- puntos extra por pickups, objetivos, contratos, Breaches y otras recompensas;
- el multiplicador de la clase seleccionada.

La fórmula conceptual es:

```
puntaje = (tiempo sobrevivido × puntos por segundo + bonos) × multiplicador de clase
```

Progresión temporal de la dificultad

En una run normal, el código abre el contenido de manera gradual:

- aproximadamente a los 30 segundos se habilitan estados especiales iniciales;
- a los 60 segundos se habilitan eventos de mapa;
- a los 90 segundos se habilita el Pulso de Contención;
- a los 150 segundos se habilitan estados de nivel 2 y State Hijack;
- a los 300 segundos se habilitan estados de nivel 3.

La clase Chaos adelanta estos tiempos y acorta muchos intervalos.

Presupuesto de presión

Los eventos grandes piden permiso antes de empezar. `GameManager` lleva una carga de presión y un límite. Si ya hay una situación pesada, otra debe esperar. Esto evita que un evento de mapa, una Breach y un estado mayor del jefe creen una combinación injusta sin control.

El director híbrido también ordena los primeros momentos importantes de la run: primero una firma propia de la arena, después un objetivo y más tarde una Breach. Luego pasa a una selección más libre.

6. `PlayerController`: controles, movimiento y defensa

`PlayerController` reúne todo lo que pertenece directamente al corredor. Lee el teclado o mouse, calcula el movimiento, administra tiempos de recarga y decide si un impacto mata, es bloqueado o es rechazado.

Controles principales

Acción	Controles
Movimiento	WASD; también flechas en modo individual
Ghost Dash	Shift
Firewall Parry	Espacio, E o clic izquierdo
Firewall Burst	Q, R o clic derecho
State Hijack	F
Pausa	Escape

El movimiento se normaliza para que avanzar en diagonal no sea más rápido que avanzar en línea recta. El jugador también es limitado al rectángulo jugable de la arena para que ningún empuje lo deje fuera del mapa.

Ghost Dash

El Dash aplica durante un instante una velocidad alta en la dirección elegida y después entra en enfriamiento. Mientras el Dash está activo, `TryAbsorbHit` considera que el impacto fue evitado. Por eso sirve tanto para desplazarse como para atravesar una amenaza en el momento correcto.

Firewall Parry

Al presionar Parry se abre una ventana muy breve y un radio alrededor del jugador. Mientras la ventana está activa, el script busca proyectiles, la anomalía o clones dentro de ese radio.

Si encuentra una amenaza:

- consume la ventana activa;
- empuja y aturde brevemente a la anomalía o al clon;
- refleja el proyectil, si era un disparo;
- agrega carga de Firewall;
- avisa al `GameManager`;
- intenta capturar el estado actual para State Hijack.

La idea importante es que presionar el botón no garantiza el Parry: el contacto debe ocurrir dentro de la ventana corta y del radio correcto.

Firewall Burst

Firewall tiene una barra propia que se carga mediante datos, powerups, nodos, peligros convertidos y Parries. Cuando llega al máximo aparece un indicador amarillo orbitando al jugador.

Al activarlo:

- la barra vuelve a cero;
- se buscan amenazas dentro de un radio amplio;
- los proyectiles son reflejados;
- la anomalía y los clones son empujados y aturdidos;
- se reproduce el efecto y se actualizan telemetría, contratos y logros.

Parry y Burst no comparten recarga. Parry es una defensa frecuente de precisión; Burst es un recurso acumulado de control de área.

Escudo

El escudo absorbe un impacto y se rompe. No vuelve al jugador invencible durante toda su duración: permanece disponible hasta vencer su tiempo o consumir el golpe.

Powerups

Powerup	Resultado
Speed Burst	Aumenta temporalmente la velocidad.
Shield	Permite absorber un impacto.
Compact	Reduce el tamaño del jugador para pasar por huecos estrechos, pero también reduce su velocidad.
Phase Dash	Durante varios segundos, cada Dash vuelve intangible el collider y permite atravesar paredes y espinas mientras dura ese Dash.

Todos los powerups también aportan carga de Firewall. Sus temporizadores se exponen al HUD para mostrar cuánto efecto queda.

Mejoras temporales

Durante una run se ofrecen elecciones que pueden mejorar velocidad, ventana, recarga y radio de Parry, duración de escudo, carga, radio o stun de Firewall, resistencia a slows y empujes, o combinaciones avanzadas. Estas mejoras modifican valores del `PlayerController`, pero solo para la run actual.

Efectos visuales sin cambiar el gameplay

El jugador se estira levemente al moverse. Para que eso no cambie injustamente el área de colisión, el código compensa el tamaño del collider. Las skins, patrones, estelas, halos y destellos también son visuales: no alteran estadísticas por sí mismos.

7. `EnemyController`: la anomalía

`EnemyController` es el script más grande porque combina movimiento, navegación, selección de estados, ataques, colisiones, recuperación ante atascos y efectos.

La anomalía siempre conserva una persecución base. Los estados no reemplazan todo su cerebro: modifican su objetivo, velocidad o ataque durante un tiempo y luego el sistema elige otro estado.

Navegación explicada sin tecnicismos

La anomalía no se limita a moverse en línea recta hacia el jugador. El proceso general es:

1. calcula un objetivo estratégico, que puede ser la posición actual, una intercepción futura o un flanco;
2. revisa si hay camino directo;
3. si hay obstáculos, divide el mapa en pequeñas celdas transitables y busca una ruta;
4. sigue puntos intermedios de esa ruta;
5. mira algunos puntos por adelantado para suavizar esquinas;
6. prueba direcciones cercanas y elige la que tenga espacio sin alejarse demasiado del objetivo;
7. reconstruye la ruta cuando cambia la geometría.

El algoritmo de grilla corresponde a una búsqueda A*: intenta encontrar un recorrido corto entre celdas libres. Encima de esa ruta se agrega un steering local, que evita que el enemigo choque contra objetos móviles o tiemble al doblar.

Recuperación ante atascos

El código compara cuánto se movió la anomalía y si realmente redujo la distancia al objetivo. Si durante varios chequeos está cerca de un obstáculo y no progresa, primero intenta una salida lateral y reconstruye la ruta. Si varios intentos fallan, entra temporalmente en Destroyer para abrir paso. Esta contramedida es una última salida, no el movimiento normal.

Selección de estados

Los estados se eligen con pesos, no con una secuencia fija. Antes del sorteo se consideran:

- el tiempo sobrevivido;
- el nivel de estados desbloqueado;
- la fase de ritmo actual;
- los estados usados recientemente;
- el evento que está activo;
- el nivel de sector;
- la clase elegida;
- enfriamientos especiales, como el de Weave Hunter.

El ritmo alterna fases de crecimiento, pico, caída y descanso. Por eso el jefe no debería encadenar únicamente ataques mayores.

Estados de la anomalía

Estado	Explicación simple
Base Pursuit	Persecución principal. Alterna persecución directa, intercepción y flanqueo mientras usa la navegación.
Split	Crea un segundo perseguidor que intenta cerrar otra ruta; luego ambos vuelven a fusionarse.
Expansion Shoot	Dispara anillos de proyectiles que se expanden y dejan huecos para cruzar.
Speed Surge	Aumenta mucho la velocidad durante un período corto.
Weave Hunter	Persigue con giros y cambios de dirección rápidos. Tiene una presencia muy baja y un enfriamiento largo.
Destroyer	Rompe una cantidad limitada de obstáculos y abre una ruta más directa. También puede usarse como recuperación de atasco.
Phase Blink	Marca una posición y luego se teletransporta hacia ella.
Pincer Barrage	Dispara desde dos frentes para cerrar al jugador en forma de pinza.
Signal Jam	Genera una descarga que vuelve menos confiable la lectura y respuesta de los controles.
Orbit Barrage	Crea una descarga de proyectiles alrededor de la anomalía.
Replay Predator	Convierte parte del recorrido reciente del jugador en una amenaza que vuelve a recorrerlo.
Checksum Lattice	Presenta nodos que deben tocarse en un orden visible. Fallar o demorarse aplica la consecuencia del estado.
Input Desync	Altera temporalmente la respuesta de las direcciones del jugador.
Map Recompile	Marca y recompila sectores del mapa, cambiando posiciones o volviendo peligrosa una franja.
Signal Possession	Crea una señal o señuelo que amenaza la zona cercana y obliga a separarse.
Phase Contract	Impone una condición temporal; cumplirla evita el castigo.
Adaptive Countermeasure	Observa tendencias de movimiento y adapta su respuesta a patrones repetidos.
Signal Tether	Une anomalía y jugador con una señal que debe cortarse usando distancia y obstáculos.
Blindspot Protocol	Protege un sector alrededor de la anomalía y deja otro como punto débil.

Los diez estados desde Phase Blink hasta Phase Contract forman el grupo de nivel 2. Los tres últimos son de nivel 3 y su lógica vive principalmente en `EnemyLevelThreeStateController` para no seguir agrandando el bucle principal.

`Topology Fold` no es un estado independiente de la lista. Cuando el nivel 3 está habilitado, puede agregarse como modificador a ciertos estados con proyectiles: enlaza bordes opuestos y transporta entidades o disparos de un lado al otro.

Daño y contacto

Cuando la anomalía o un proyectil toca al jugador, el orden conceptual es:

1. comprobar si hay un Parry activo;
2. si no, comprobar si Dash o escudo absorben el impacto;
3. si ninguna defensa responde, pedir la derrota al `GameManager`.

Esto mantiene una sola regla de resolución aunque el golpe provenga de objetos distintos.

8. State Hijack: convertir defensa en contraataque

`PlayerStateHijack` está separado de `PlayerController` porque tiene una tarea concreta: guardar una habilidad robada y ejecutarla una vez.

Después de que el sistema se desbloquea, un Parry exitoso contra la anomalía puede capturar el estado actual. El código agrupa distintos estados en seis respuestas útiles:

Habilidad robada	Qué hace
Overclock	Aumenta temporalmente la velocidad.
Phase Step	Teletransporta al jugador en la dirección elegida buscando un destino libre.
Echo Decoy	Deja un señuelo que distrae a la anomalía.
Orbit Guard	Crea nodos orbitales que reflejan proyectiles cercanos.
Breach Cutter	Rompe obstáculos frente al jugador; si no encuentra uno, frena brevemente a la anomalía.
Containment Lock	Inmoviliza temporalmente a la anomalía.

Solo se guarda una habilidad a la vez y se consume al presionar F. La mecánica transforma una buena defensa en una decisión ofensiva posterior.

9. Generación procedural y sectores

`ProceduralArenaGenerator` puede crear cinco temas:

Tema	Regla principal
Lab	Seguridad reactiva: escáneres, barreras y sistemas de contención.
Storage	Logística viva: grúas y carga redibujan rutas.
Rupture	Espacio inestable: órbitas, grietas, portales y ecos.
Core	Red programable: relés y compuertas que el jugador puede controlar.
Archive	Gravedad corrupta: campos que atraen y hacen orbitar entidades.

La primera zona disponible tras borrar el progreso es Lab. Al atravesar sectores se descubren y guardan nuevos temas. La progresión de una run permite Lab y Storage en sectores tempranos, Rupture desde el sector 3, Core desde el 4 y Archive desde el 5.

Cada Breach incrementa `SectorLevel`. Ese nivel aumenta la presión y las recompensas, comunica el nuevo sector al enemigo y favorece temas todavía no visitados durante esa run.

Los obstáculos dinámicos usan `DynamicObstacleController`, que puede configurarlos para deslizarse, pulsar de tamaño o moverse en órbitas. Rupture usa una distribución concéntrica especial para que su evento de rotación tenga una estructura legible.

10. Eventos globales y eventos por mapa

`ArenaChaosDirector` controla contenido que puede aparecer en cualquier tema:

- Powerups.
- Pickups de puntaje.
- Data Cores, que valen más y cargan Firewall.
- Pulso de Contención.
- Nodos de Sincronización.
- Breach.

Eventos globales

Evento	Funcionamiento
Pulso de Contención	Círculos amarillos avisan zonas que luego se vuelven rojas. Dentro de ellas el jugador se ralentiza y la anomalía se acelera.
Nodos de Sincronización	El jugador debe permanecer sobre todos los nodos antes del límite para obtener puntaje.
Breach	Portal de escape seguido por un barrido. Entrar a tiempo regenera la arena y avanza de sector; quedar atrás termina la run.

Lab

Evento	Funcionamiento
Barrido de Esterilización	Bandas amarillas anticipan el barrido rojo; los corredores verdes son zonas seguras.
Grilla de Seguridad	Líneas que se activan y perjudican al jugador mientras favorecen a la anomalía.
Terminales de Contención	Hay que cargar todas las terminales para encerrar temporalmente a la anomalía y obtener Firewall.
Cierre de Seguridad	Un escáner anuncia el corredor que dos compuertas cerrarán desde los bordes.

Storage

Evento	Funcionamiento
Transportadores	Los carriles empujan al jugador, la anomalía y objetos en la dirección indicada.
Reorden Magnético	Las grúas reubican obstáculos y cambian los pasillos disponibles.
Ruta de Carga	Una formación de bloques cruza el sector y debe atravesarse usando sus huecos.
Grúa de Carga	El sistema ambiental mueve, agrega o retira carga fuera de los eventos principales.

Rupture

Evento	Funcionamiento
Ruptura Rotativa	Los objetos orbitan alrededor del centro y abren o cierran huecos.
Cadena de Ecos	Tocar dos anclas crea una línea; atraer a la anomalía para que la cruce la aturde y carga Firewall.
Portales de Eco	El jugador entra por un portal, sale por otro y deja una trampa para la anomalía.
Fragmentos de Ruptura	Grietas anuncian obstáculos temporales que se materializan y luego desaparecen.

Core

Evento	Funcionamiento
Red Reprogramable	Se activa el relé verde correcto en secuencia; cada acierto recompila las compuertas y completar la red aísla a la anomalía.
Protocolo de Espinas	Algunos obstáculos de Core pasan de amarillo a rojo y se vuelven dañinos. Phase Dash permite atravesarlos durante el Dash intangible.

Archive

Evento	Funcionamiento
Pozo Gravitacional	Núcleos móviles atraen al jugador, la anomalía, proyectiles, clones y pickups hasta curvar su movimiento en órbitas.

Cómo se implementa un evento

Aunque cada evento es distinto, suelen compartir este ciclo:

1. esperan su intervalo;
2. consultan si la run y el mapa permiten activarse;
3. reservan presupuesto de presión;
4. muestran una advertencia amarilla;
5. activan la regla o peligro rojo;
6. comunican un texto breve al HUD y al tutorial contextual;
7. restauran la arena y liberan la presión al terminar.

El contrato `IThemedEventStatusProvider` permite que `GameManager` pregunte qué evento está activo sin saber si la arena actual es Lab, Storage, Rupture, Core o Archive.

11. Contratos y mejoras durante la run

Contratos

Los contratos son objetivos temporales creados por `GameManager`. Pueden pedir:

- sobrevivir cierta cantidad de tiempo;
- sumar puntos;
- recoger pickups;
- realizar Parries;
- activar Firewall Burst.

Si se completan antes del límite entregan puntos y datos adicionales. Si fallan, desaparecen y el siguiente contrato se programa más adelante. No tienen un tutorial propio por contrato porque todos comparten la misma lectura de meta, progreso y tiempo.

Mejoras

En momentos determinados la run se pausa y aparecen opciones aleatorias. El jugador elige una y el código llama al método correspondiente de `PlayerController`. Algunas mejoras están disponibles desde el inicio y otras deben comprarse con Datos en el Archivo.

La selección tiene un límite de tiempo que se acorta en etapas avanzadas. Elegir una mejora también suma un pequeño bono de puntaje y se registra en telemetría.

12. Clases o estilos de juego

`ContainmentOperationStorage` guarda la clase seleccionada antes de cargar gameplay. La clase no es solo texto: `GameManager`, `PlayerController`, `EnemyController` y `ArenaChaosDirector` aplican sus modificadores.

Clase	Ventaja	Desventaja o presión
Normal	Sistemas sin modificar.	No tiene bonificación especial.
Attack	Más velocidad, carga de Firewall y radio de Parry; multiplicador de puntaje.	La anomalía es más rápida y cambia antes de estado.
Defend	Parry más rápido y amplio, mejor resistencia ambiental.	Menor velocidad de movimiento.
Achiever	Multiplicador alto de puntaje.	No aparecen powerups.
Chaos	Mucho más puntaje y Datos; más powerups, pickups, contratos y mejoras.	Todo escala antes y sucede con mayor frecuencia; la anomalía también es más rápida.

La clase Chaos modifica deliberadamente casi todos los relojes importantes. Es una versión acelerada de la experiencia completa, no solo “más eventos”.

13. Tutoriales

Hay dos capas:

Tutorial inicial

Presenta de forma ordenada:

1. movimiento con WASD;
2. Parry con teclado o mouse;
3. Firewall;
4. recursos y puntaje;
5. eventos de arena;
6. confirmación para iniciar.

Tutorial contextual

Observa lo que ocurre durante gameplay. Se abre cuando una mecánica realmente está disponible o aparece por primera vez: Dash, Parry cercano, Firewall lleno, pickup, powerup, mejora, State Hijack, estado especial, Breach o evento de mapa.

Cada evento tiene una clave estable. `GameManager` mantiene una cola de tutoriales pendientes para que dos eventos simultáneos no abran paneles uno encima del otro. `UserSettings` guarda qué claves ya fueron vistas, por lo que cada explicación aparece una sola vez hasta reiniciar tutoriales o borrar el progreso.

14. Interfaz, jerarquía visual y colores

La mayor parte de la UI usa `OnGUI`. Los controladores calculan rectángulos, escalas y estilos según la pantalla. `GlobalFontSettings` entrega las fuentes compartidas y `UserSettings` entrega la escala elegida para menú y HUD.

`GlitchUiPalette` define un código de color semántico:

Color	Significado
Cian	Información, navegación y mecánicas de control.
Amarillo	Advertencia o algo que está por activarse.
Rojo	Peligro ya activo, daño o derrota.
Verde	Éxito, recompensa u objetivo favorable.
Violeta	Recurso o habilidad especial.
Blanco azulado	Información neutral.

La regla que se puede explicar al profesor es: “el color informa antes de decorar”. Por ejemplo, una zona amarilla todavía permite reaccionar; cuando cambia a rojo, la amenaza ya está activa.

El HUD principal reúne tiempo, puntaje, sector, estado del jefe, Dash, Parry, Firewall, State Hijack, powerups, contrato y evento activo. No todos los datos tienen el mismo peso: las amenazas temporales aparecen como alertas y los recursos permanentes quedan en barras estables.

`GameMenuController` oculta el HUD al pausar o perder. La pantalla de derrota calcula automáticamente el rango:

Rango	Puntaje mínimo
S	3500
A	2200
B	1200
C	500
D	Menos de 500

15. Audio

`GlitchAudioManager` es una fachada: los demás scripts piden una intención, como `PlayParrySuccess`, sin buscar clips ni manejar volúmenes.

El manager:

- sobrevive a los cambios de escena;
- separa música y efectos;
- respeta volumen maestro, música y SFX;
- cambia entre música de menú y gameplay;
- agrega capas de intensidad según la amenaza;
- refuerza momentos como Firewall, Breach y despertares del jefe;
- puede generar sonidos procedurales si no existe un clip externo.

Esto evita repetir configuración de audio en cada mecánica.

16. Guardado local y metaprogresión

Los archivos terminados en `Storage` suelen ser clases estáticas. No necesitan existir como objetos visibles: otros scripts les preguntan o les envían datos.

`MetaProgressionStorage`

Guarda:

- Datos disponibles;
- runs y puntuación acumuladas;
- mejores puntajes y tiempos;
- récord por zona;
- módulos de mejora desbloqueados;
- skins compradas;
- skin seleccionada;
- resumen de la última run.

Al terminar, calcula Datos a partir del puntaje y la supervivencia, suma el bono de contratos y actualiza los récords.

`AchievementStorage`

Contiene el catálogo de logros, contadores acumulados y recompensa. Un logro se desbloquea una sola vez y puede premiar Datos.

`DailyChallengeStorage`

Usa la fecha para elegir siempre el mismo desafío durante ese día. Puede pedir supervivencia, puntos, pickups, Parries, Bursts o contratos. Al completar la meta entrega Datos una sola vez.

`ArenaDiscoveryStorage`

Guarda el orden en que se descubrieron los temas. Esto permite que los mapas avanzados entren en la selección de futuras runs.

`UserSettings`

Guarda volumen maestro, música, SFX, escala de menú, escala del HUD, intensidad de movimiento visual, pantalla completa, VSync y progreso de tutoriales.

Borrar progreso

El botón de Opciones usa una confirmación de dos pasos. Borra metaprogresión, logros, desafío diario, caché y mejor marca local, nombre, clase, descubrimiento, tutoriales, opciones de debug y telemetría. Conserva las preferencias audiovisuales y la identidad anónima de Firebase. Después vuelve a pedir un nombre.

17. Ranking global con Firebase

El ranking está dividido en dos partes:

- `RankingStorage` prepara, valida, ordena y guarda una caché local;
- `FirebaseLeaderboardService` se comunica con Firebase.

Flujo de un puntaje

1. La pantalla de derrota toma el nombre registrado y el resultado.
2. `RankingStorage` compara el puntaje con la mejor marca personal conocida.
3. Si es menor o igual, no realiza una escritura global.
4. Si es mayor, guarda el resultado como pendiente y llama al servicio.
5. Firebase autentica de forma anónima y lee la entrada del usuario.
6. Solo si el nuevo puntaje también supera al remoto se realiza el reemplazo.
7. Después se actualiza la lista de los mejores 15.

La lectura general usa caché durante un tiempo para no consultar la red en cada cuadro. La escritura usa un ETag: primero obtiene la versión actual y luego intenta reemplazar exactamente esa versión. Así se reduce el riesgo de sobrescribir un cambio inesperado.

La identidad de Firebase y el nombre visible cumplen funciones distintas. Firebase usa un usuario anónimo interno para la seguridad; el jugador registra una sola vez el nombre que se muestra en la tabla.

18. Telemetría local

`RunTelemetryStorage` abre un registro al comenzar una run y acumula todo en memoria. Solo escribe al finalizar, lo que evita acceder al disco por cada pickup o Parry.

Por cada run guarda:

- versión del juego y un identificador de sesión;
- fecha de inicio y fin;
- mapa y clase;
- puntaje y supervivencia;
- cantidad de pickups, powerups, Parries y Bursts;
- Breaches, contratos y mejoras;
- motivo de finalización: derrota o abandono.

También mantiene un resumen acumulado con mejores valores y totales. Los archivos se guardan dentro de la carpeta persistente de Unity, en `telemetry/runs.jsonl` y `telemetry/summary.json`. No se envían a Firebase.

19. Duelo local

El modo local reutiliza al jugador, la anomalía, la arena y los eventos, pero agrega `LocalVersusManager`.

- Jugador 1 controla al corredor con WASD y sus habilidades.
- Jugador 2 mueve la anomalía con flechas y puede cambiar de estado con Control derecho o Enter cuando el cambio está habilitado.
- Ambos deben confirmar que están listos.
- La ronda dura 120 segundos.
- Si la anomalía toca al corredor sin defensa, gana la anomalía.
- Si se termina el tiempo, gana el corredor.

El Parry del corredor usa cargas limitadas en este modo y se recarga como conjunto. La interfaz reserva laterales distintos para mostrar recursos de cada jugador y el reloj en el centro.

20. Menús y cambios de escena

`MainMenuController`

Presenta datos y convierte clics en acciones. No calcula por sí mismo la progresión ni el ranking: consulta a los Storage. También dibuja el fondo animado, las partículas orbitales y las transiciones entre paneles.

`GameMenuController`

Tiene tres estados simples: jugando, pausa y derrota. Cambia `Time.timeScale` para detener o continuar, controla el cursor, oculta el HUD y ofrece continuar, reiniciar o volver al menú. En derrota muestra el rango y registra automáticamente la marca.

`SceneTransitionController`

Es una instancia persistente que cubre la pantalla, carga la escena de forma asíncrona y vuelve a descubrirla con un fundido. También bloquea clics mientras la pantalla está cubierta, evitando dobles entradas durante la transición.

21. Inventario de scripts por grupo

No hace falta memorizar todos. Este inventario sirve para saber dónde buscar.

Núcleo de gameplay

- `GameManager.cs`: coordinación completa de la run y HUD.
- `PlayerController.cs`: controles, movimiento, defensa, powerups y visuales del jugador.
- `PlayerStateHijack.cs`: captura y uso de habilidades robadas.
- `EnemyController.cs`: persecución, estados, ataques, navegación y daño.
- `EnemyLevelThreeStateController.cs`: estados complejos de nivel 3 y Topology Fold.
- `SplitAnomalyCloneController.cs`: clon temporal del estado Split.
- `AnomalyProjectile.cs`: movimiento, colisión y reflejo de proyectiles.

Arena y contenido global

- `ProceduralArenaGenerator.cs`: construcción y cambio de temas.
- `ArenaChaosDirector.cs`: powerups, datos, objetivos, pulsos y Breach.
- `DynamicObstacleController.cs`: movimiento de obstáculos.
- `ContainmentHazardZone.cs`: zonas con aviso y efecto activo.
- `ArenaPowerupPickup.cs`, `ArenaScorePickup.cs` y `ArenaObjectiveNode.cs`: objetos recolectables e interactivos.

Eventos temáticos

- `LabSweepEventController.cs` y `LabAmbientSecurityController.cs`.
- `StorageSurgeEventController.cs` y `StorageAmbientCraneController.cs`.
- `RuptureSpinEventController.cs` y `RuptureAmbientRiftController.cs`.
- `CoreGateEventController.cs` y `CoreSpikeEventController.cs`.
- `ArchiveNullFieldController.cs`.

Los scripts de terminales, portales, ecos, compuertas, carga y espinas son piezas auxiliares creadas por esos controladores.

Menú e interfaz

- `MainMenuController.cs`: menú principal.
- `GameMenuController.cs`: pausa y derrota.
- `LocalVersusManager.cs`: presentación y reglas del duelo.
- `GlitchUiPalette.cs`: colores semánticos.
- `GlobalFontSettings.cs`: fuentes compartidas.
- `SceneTransitionController.cs`: fundidos y carga.

Guardado y servicios

- `MetaProgressionStorage.cs`: economía, estadísticas, módulos y skins.
- `AchievementStorage.cs`: logros.
- `DailyChallengeStorage.cs`: operación diaria.
- `ArenaDiscoveryStorage.cs`: mapas descubiertos.
- `ContainmentOperationStorage.cs`: clase elegida.
- `RankingStorage.cs`: datos y caché del ranking.
- `FirebaseLeaderboardService.cs`: red y autenticación.
- `RunTelemetryStorage.cs`: datos locales de sesiones.
- `UserSettings.cs`: preferencias.
- `DeveloperModeStorage.cs`: opciones de prueba ocultas.

Audio, sprites y efectos

- `GlitchAudioManager.cs`: música y SFX.
- Los archivos terminados en `SpriteProvider` generan y reutilizan figuras simples para iconos y efectos.
- Los archivos terminados en `Fx` dibujan una animación concreta y luego desaparecen. Normalmente no deciden reglas ni recompensas.

22. Ejemplo de interacción completa

Supongamos que el jugador recoge un powerup de Phase Dash:

1. `ArenaChaosDirector` decide que corresponde crear un powerup y encuentra una posición libre.
2. Crea un objeto con sprite, collider y `ArenaPowerupPickup`.
3. El jugador entra en su collider.
4. El pickup llama a `PlayerController.ApplyPhaseDashPowerup`.
5. `PlayerController` inicia el temporizador y el HUD puede consultarlo.
6. El pickup agrega carga de Firewall.
7. Avisa a `GameManager`, que actualiza tutorial y telemetría.
8. Avisa al director, que entrega puntos y libera la referencia del pickup.
9. El objeto reproduce sonido, crea un efecto y se destruye.
10. Mientras el temporizador siga activo, cada Ghost Dash convierte el collider del jugador en trigger durante ese impulso y permite atravesar paredes o espinas.

Este ejemplo muestra el patrón general del proyecto: un objeto detecta la acción, el dueño de la mecánica aplica el efecto y el manager recibe un aviso para los sistemas secundarios.

23. Qué conviene decir si preguntan por decisiones de diseño del código

“¿Por qué hay un GameManager?”

Porque muchos sistemas necesitan compartir el estado general de la run. Centralizar comienzo, tiempo, dificultad y final evita que cada enemigo o pickup invente su propia versión de si la partida está activa.

“¿Por qué el jugador y el enemigo tienen métodos públicos?”

Para ofrecer acciones concretas sin exponer toda su información interna. Un evento puede pedir `ApplyMovementSlow` sin saber cómo se guardan los temporizadores del jugador.

“¿Por qué hay controladores distintos para cada mapa?”

Porque la regla de Lab no debería mezclarse con la de Rupture. El generador activa solo los componentes del tema actual y el HUD los consulta mediante una interfaz común.

“¿Por qué se separó el nivel 3 del enemigo?”

Porque los estados avanzados tienen lógica extensa. Separarlos reduce la responsabilidad del bucle principal y permite que `EnemyController` conserve la navegación general.

“¿Cómo se evita una dificultad injusta?”

Con progresión temporal, fases de ritmo, filtros de estados, enfriamientos, señales visuales y un presupuesto global que limita amenazas simultáneas.

“¿Cómo se optimiza Firebase?”

La lista se mantiene en caché, se solicitan solo 15 entradas y solo se escribe cuando el puntaje supera la mejor marca personal local y remota.

“¿Cómo se optimiza la telemetría?”

Los contadores viven en memoria durante la run. Se hace una escritura del registro y una del resumen cuando termina, en lugar de escribir por cada acción.

24. Guion oral de cinco minutos

Podés usar esta versión casi textual:

GUION SUGERIDO

GLITCH es un juego de supervivencia donde un corredor escapa de una anomalía dentro de arenas procedurales. Organicé el código por responsabilidades. El GameManager dirige la run: controla el comienzo, el tiempo, el puntaje, la dificultad, tutoriales, contratos, mejoras y el final. El PlayerController concentra los controles y decide cómo responde el jugador a cada impacto. El EnemyController maneja la persecución, la navegación y una máquina de estados que va habilitando ataques más complejos.

La arena no está fija. ProceduralArenaGenerator construye paredes, obstáculos, detalles y controladores según cinco temas. ArenaChaosDirector agrega contenido compartido, como datos, powerups, nodos, pulsos y Breaches. Cada mapa tiene además eventos propios: Lab usa seguridad y terminales; Storage usa grúas y carga; Rupture usa órbitas, portales y ecos; Core usa relés y espinas; Archive altera la gravedad.

Los sistemas se comunican principalmente mediante avisos y métodos públicos. Por ejemplo, cuando hago un Parry, PlayerController resuelve el contacto y le avisa a GameManager. Entonces el mismo hecho puede cargar Firewall, avanzar un contrato, un logro, un desafío y la telemetría sin poner toda esa lógica dentro del enemigo.

La dificultad se regula en capas. Los estados se desbloquean por tiempo, la anomalía usa fases de ritmo y un sorteo con pesos, y los eventos reservan presupuesto de presión para evitar demasiadas amenazas grandes juntas. Si la anomalía se atasca, primero recalcula la ruta y busca una salida; Destroyer queda como contramedida final.

Al terminar se calcula el rango S a D, se entregan Datos y se actualiza la progresión. La telemetría se guarda localmente con una sola escritura al final. El ranking usa Firebase con usuario anónimo, caché y actualización exclusiva del récord personal. De esa manera el gameplay, la presentación y el guardado quedan conectados, pero cada sistema conserva una responsabilidad entendible.

25. Plan breve para aprenderlo

No intentes memorizar archivos de miles de líneas. Estudiá en este orden:

1. Aprendé el flujo completo de la sección 4.
2. Memorizá la responsabilidad de los cinco componentes principales.
3. Explicá con tus palabras Parry, Firewall, navegación y Breach.
4. Elegí un evento de cada mapa y contá su ciclo de aviso, activación y cierre.
5. Practicá el ejemplo de Phase Dash de la sección 22.

6. Leé en el código los métodos `Update` de `GameManager`, `PlayerController` y `EnemyController`, pero solo para reconocer el orden de tareas.
7. Practicá las preguntas de la sección 23.

Si podés explicar por qué cada responsabilidad vive en su script y seguir una acción desde que ocurre hasta que se guarda, ya entendés la estructura importante del proyecto. No hace falta recitar nombres de variables ni valores exactos salvo que te los pregunten.